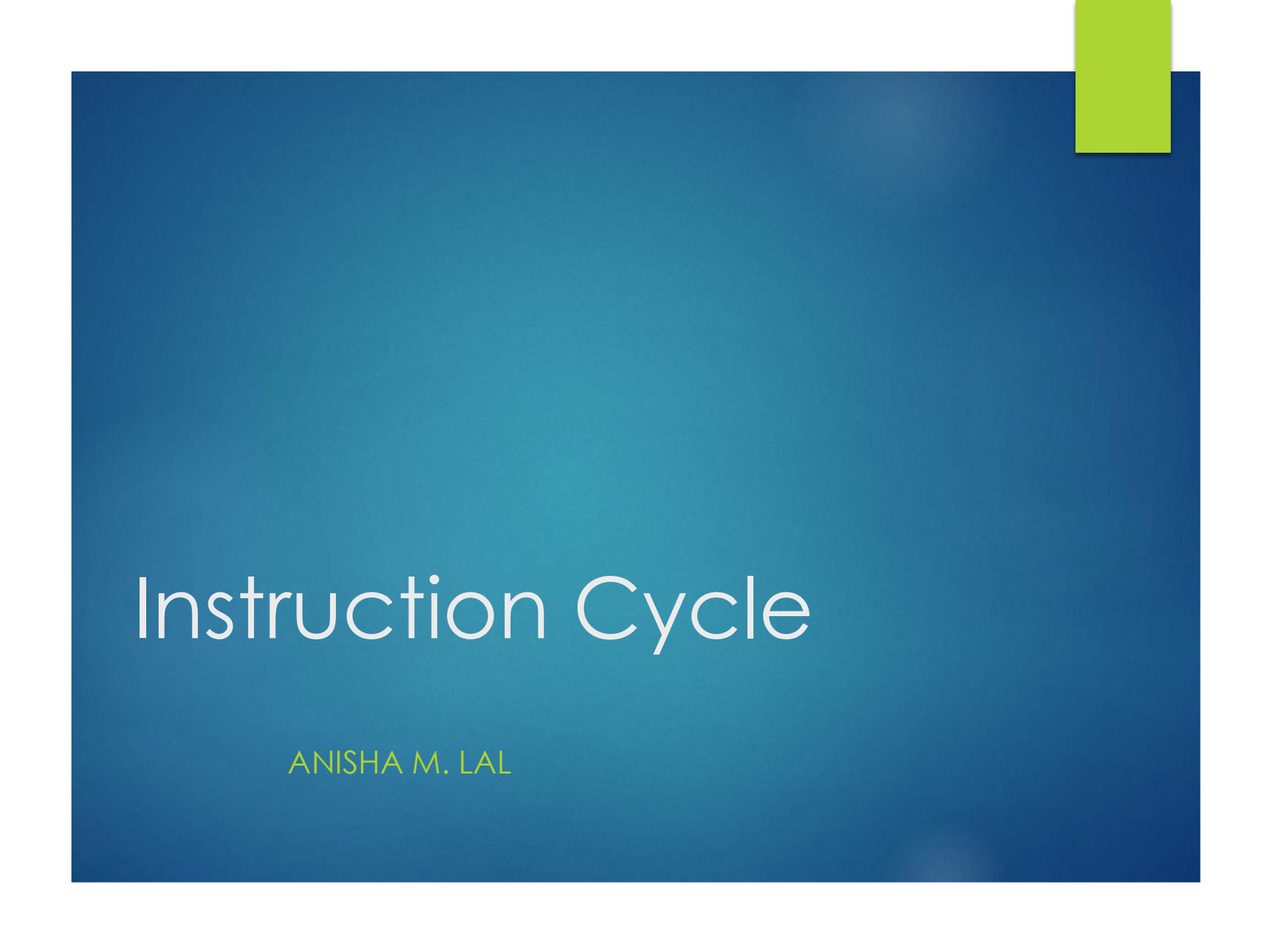




Instruction Cycle

ANISHA M. LAL



Topics

Instruction cycle.

Phases of instruction cycle

Instruction types

Subroutine call and return mechanisms.

Instruction cycle.

- ▶ Instruction Cycle
 - Steps required for processing a single instruction
- ▶ Phases of Instruction cycle:
 - Fetch cycle
 - decode cycle
 - Execute cycle
- ▶ Execute cycle has following action:
 - Processor- memory
 - Processor –I/O
 - Data Processing
 - Control

Phases of Instruction Cycle(State Diagram)

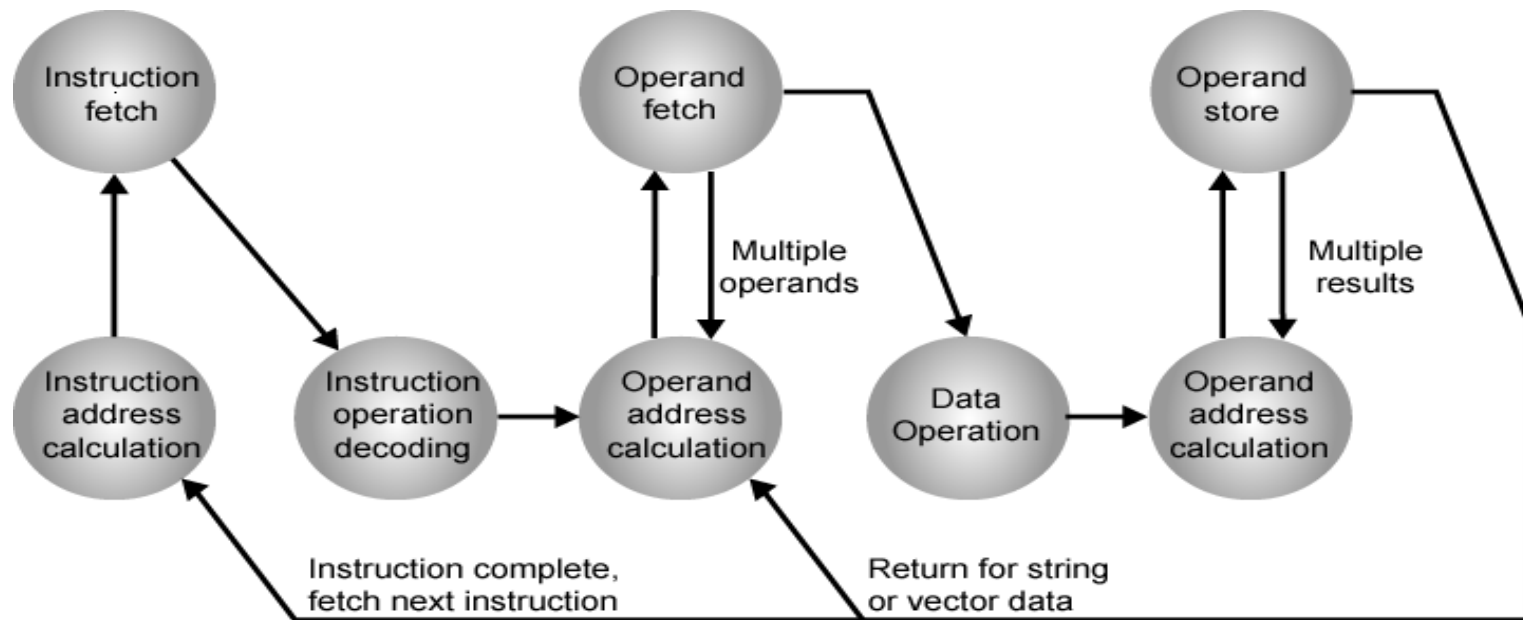


Figure 3.6 Instruction Cycle State Diagram

Phases of Instruction Cycle

- ▶ **Instruction address calculation (iac):**
 - ▶ Determine the address of the next instruction to be executed. Adding a fixed number to a next number.
- ▶ **Instruction fetch: (if)**
 - ▶ Read the instruction from its memory location into the processor.
- ▶ **Operand Address Calculation: (oac)**
 - ▶ Analyze instruction to determine type of operation to be performed and operand(s) to be used.
- ▶ **Operand Address Calculation: (oac)**
 - ▶ If the operation involves the reference to an operand in memory or available via I/O, then determine the address of the operand.
- ▶ **Operand Fetch (of):**
 - ▶ Fetch the operand from memory or read it from I/O.
- ▶ **Data Operation (do):**
 - ▶ Perform the operation indicated in the instruction.
- ▶ **Operand store (os)**
 - ▶ Write the result into memory or out to I/O.

Consider a machine in which the processor has a program counter to hold the address of instruction, Instruction register to hold instruction to be executed, and Accumulator for temporary storage. The data and instruction length are 16 bits. Instruction format provides 4 bit for opcode and 12 bit for address.

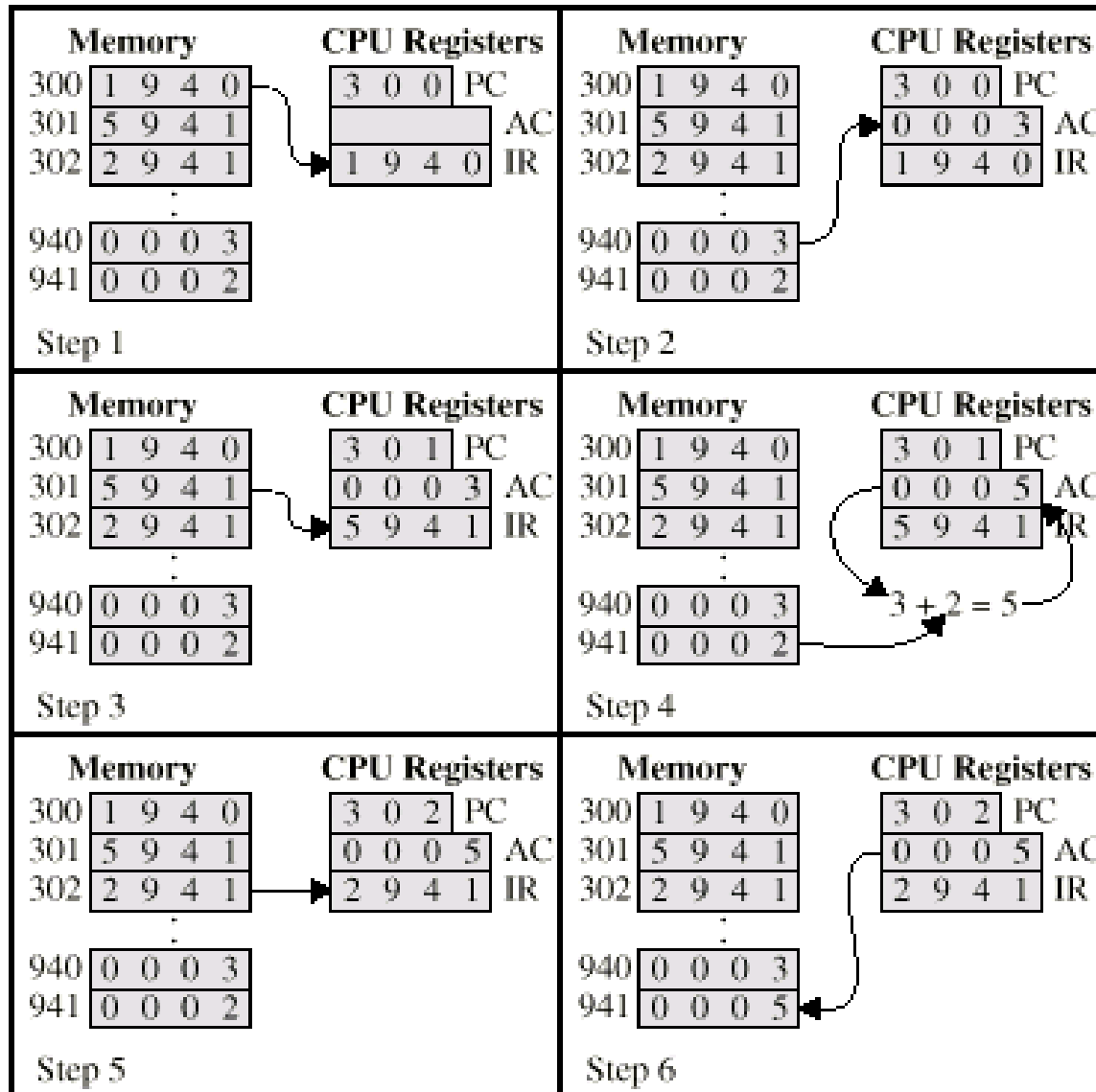
The following example describes the execution of an addition program. $[X+Y=Z]$

Hexadecimal representation of opcode and operand address is given.

Example of Program Execution



0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory



Instruction types

- ▶ Data transfer instructions
- ▶ Data manipulation instructions
 - ▶ Arithmetic instructions
 - ▶ Logical and bit manipulation instructions
 - ▶ Shift instructions
- ▶ Program control instructions

Data transfer instructions

- ▶ Move data from one place to another without changing the data content in the computer.
- ▶ Different data transfers:
 - ▶ Memory ↔ processor registers
 - ▶ Processor registers ↔ input or output
 - ▶ Processor register ↔ processor register

Set of data transfer instructions

- ▶ Load – transfer from memory to a processor register
- ▶ Store – transfer from processor register into memory
- ▶ Move – transfer from one register to another, transfer between register and memory or between two memory words.
- ▶ Exchange – swaps information between two registers or a register and a memory word
- ▶ Input – transfer data among registers/memory and input terminal
- ▶ Output – transfer data among register/memory and output terminal
- ▶ Push – transfer data from register/memory to memory stack
- ▶ Pop – transfer data from stack to register/memory

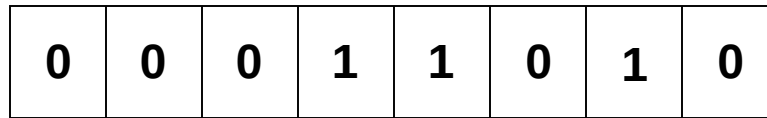
Data Manipulation Instructions

- ▶ Perform operations on data and provide the computational capabilities for the computer.
- ▶ Arithmetic instructions
 - ▶ Increment
 - ▶ Decrement
 - ▶ Add
 - ▶ Subtract
 - ▶ Multiply
 - ▶ Divide
 - ▶ Add with carry
 - ▶ Subtract with borrow
 - ▶ Negate (2's complement) – change the sign of the operand
 - ▶ Absolute – replace operand by its absolute value
 - ▶ Arithmetic shift left
 - ▶ Arithmetic shift right

► Arithmetic shift left



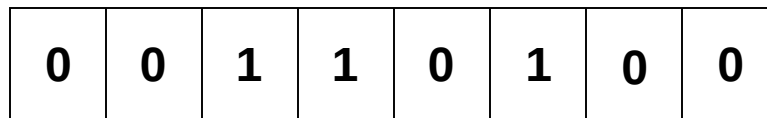
Sign bit



Sign bit

Shift by 1 bit towards left

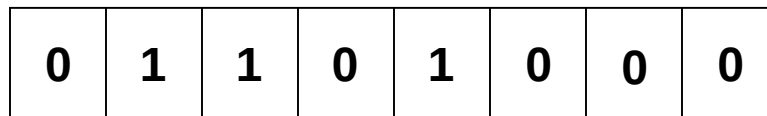
0



Sign bit

After shifting two times

0



Sign bit

After shifting three times

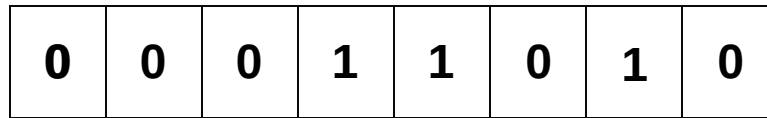
0

Overflow occurs as
sign bit changes

► Arithmetic shift Right

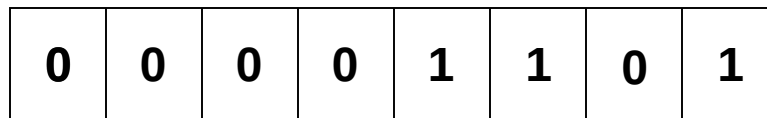


↑
Sign bit



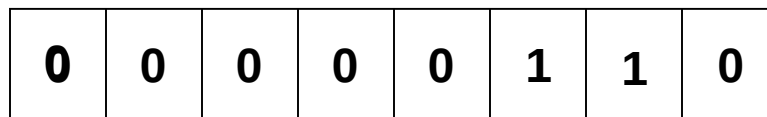
↑
Sign bit

Shift by 1 bit towards right



↑
Sign bit

After shifting two times



↑
Sign bit

After shifting three times

Data manipulation instructions

► Logical and Bit manipulation instructions

- Clear (can also be included in data transfer instruction based on the way the operation is performed – 0's transferred to the destination)
- Complement
- AND- ($x \text{ AND } 0 = 0$) to clear a bit
- OR – ($x + 1 = 1$) set a bit
- Ex-Or – ($x \text{ xor } 1 = x'$) to complement a bit
- Clear carry
- Set carry
- Complement carry
- Enable interrupt – flip-flop that controls the interrupt facility is enabled
- Disable interrupt – flip-flop that controls the interrupt facility is disabled

Data manipulation instructions

- ▶ Shift Instructions
 - ▶ Logical left shift
 - ▶ Logical right shift
 - ▶ Arithmetic shift left
 - ▶ Arithmetic shift right
 - ▶ Rotate right
 - ▶ Rotate left
 - ▶ Rotate right through carry
 - ▶ Rotate left through carry

► Logical shift left

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Shift by 1 bit towards left

0

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

After shifting two times

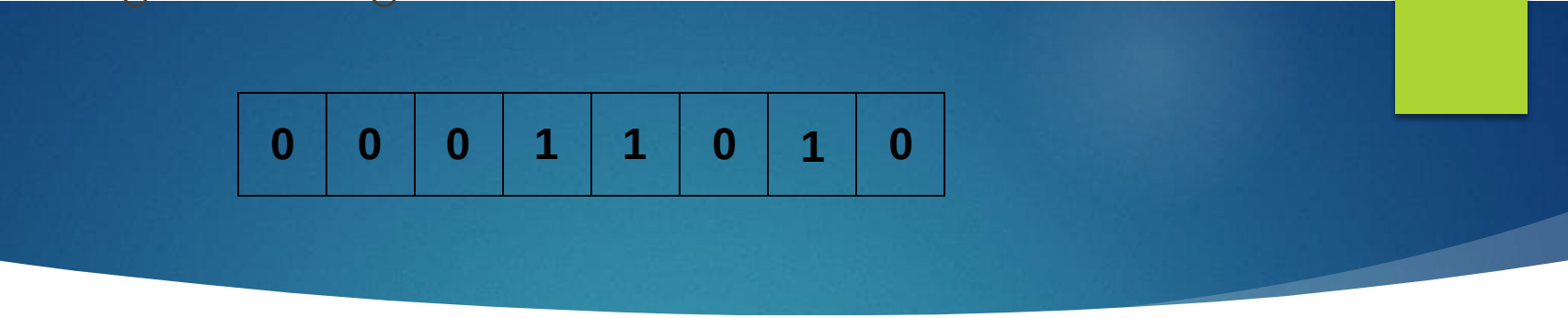
0

0	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---

After shifting three times

0

► Logical shift Right



0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

0

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Shift by 1 bit towards right

0

0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

After shifting two times

0

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

After shifting three times

► Rotate left

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Buffer

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Rotate by 1 bit towards left

Buffer

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

After rotating two times

Buffer

0	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---

After rotating three times

► Rotate right

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

rotate by 1 bit towards right

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Buffer

0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

After rotating two times

Buffer

1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

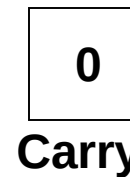
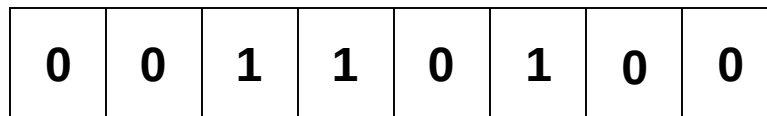
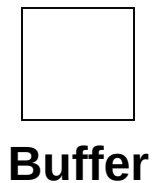
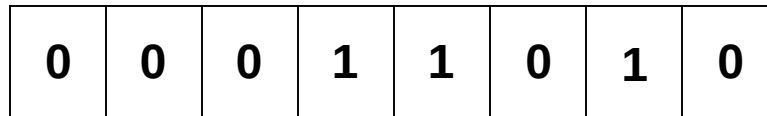
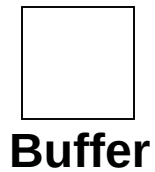
After rotating three times

Buffer

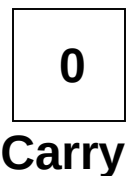
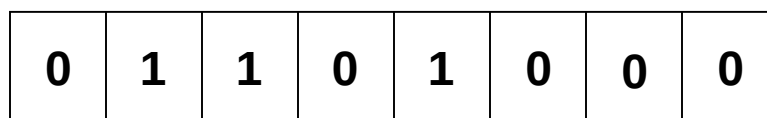
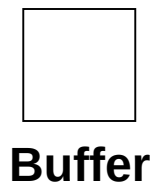
► Rotate left through carry



Rotate by 1 bit towards left



After rotating two times



After rotating three times

► Rotate right through carry

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

rotate by 1 bit towards right

0

Carry

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

--

Buffer

0

Carry

0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---

--

Buffer

After rotating two times

0

Carry

1	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

--

Buffer

After rotating three times

Program control Instructions

- ▶ Branch
- ▶ Jump
- ▶ Skip
- ▶ Call
- ▶ Return
- ▶ Compare (by subtraction)

Subroutine

- ▶ Subroutine is a self-contained sequence of instructions that performs a given computational task.
- ▶ It may be called many times at various points in the main program
- ▶ When called, branches to 1st line of subroutine and at the end, returned to main program.
- ▶ Different names to the instruction that transfers program control to a subroutine
 - ▶ Call subroutine
 - ▶ Jump to subroutine
 - ▶ Branch to subroutine
 - ▶ Branch and save address

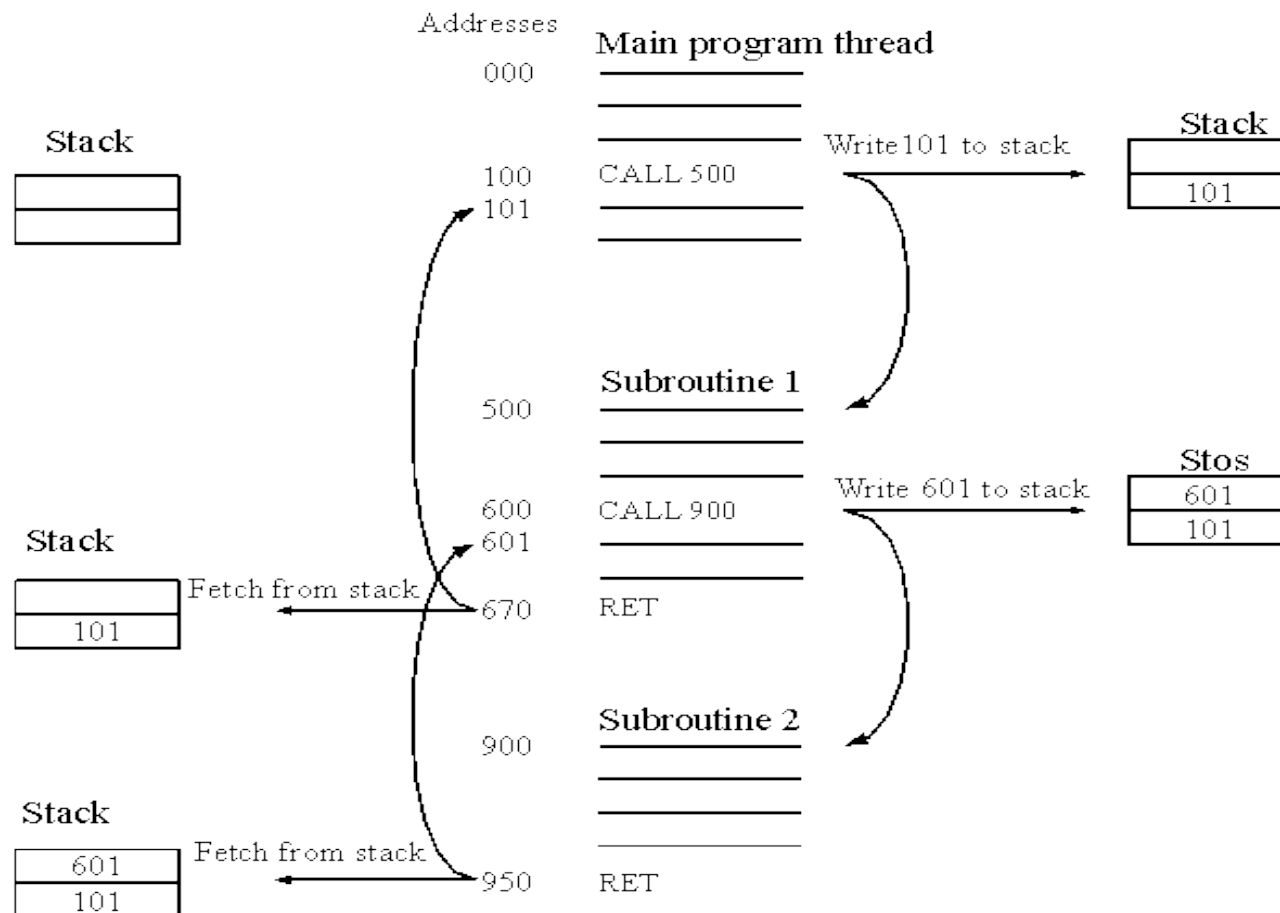
Control transfer from called to caller

- ▶ Subroutine instruction – Opcode + starting address of the subroutine
- ▶ Execution:
 - ▶ PC content (return address) is stored in a temporary location
 - ▶ Control is transferred to the subroutine
- ▶ when return
 - ▶ Transfers the return address from the temporary location to the PC.
 - ▶ Control is transferred back to the called routine

Locations to store the return address

- ▶ First memory location of the subroutine
- ▶ Fixed location in memory
- ▶ Processor registers
- ▶ Memory stack – best option
 - ▶ Adv: In the case of sequential calls to subroutines. So, the top of the stack always has the return address of the subroutine which to be returned first.

Main memory



Micro-operations

Call:

$SP \leftarrow SP - 1$ // decrement stack pointer

$M[SP] \leftarrow PC$ // push content of PC onto the stack

$PC \leftarrow \text{effective address}$ /* transfer control to the
subroutine */

Return:

$PC \leftarrow M[SP]$ // pop stack and transfer to PC

$SP \leftarrow SP + 1$ // increment stack pointer

Recursive subroutines

- ▶ Subroutine that calls itself
- ▶ If only one register or memory location is used to hold the return address, when subroutine is called recursively, it destroys the previous return address.
- ▶ So, stack is the good solution for this problem